

Reviewer Pack — Schur-Weyl Multiplicity Gap in Permanent Communication Matri...

Entry 39e42593cacd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 02:29:35 UTC

Schur-Weyl Multiplicity Gap in Permanent Communication Matrices

Entry ID: 39e42593cacd **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 02:29:35 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl duality **Field B** (complexity object): Communication matrix rank

Statement:

For random 3-CNF formulas with n variables, the multiplicity of the symmetric square representation in the decomposition of the permanent's communication matrix exceeds that of the determinant's matrix by $\Omega(2^n)$.

Rationale (proposer's reasoning):

Schur-Weyl duality decomposes tensor spaces into irreducible representations; permanent's higher multiplicity in symmetric squares could reflect its inherent symmetry complexity, creating a gap exploitable for communication lower bounds.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e5dc7a4b8c789436

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0 for _ in range(n)]
    for i in range(n-1, -1, -1):
        sum_val = 0
        for j in range(i+1, n):
            sum_val += A[i][j] * x[j]
```

```

        x[i] = (b[i] - sum_val) / A[i][i]
    return x

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        sign = (-1) ** (j % 2)
        det += sign * A[0][j] * determinant(submatrix)
    return det

def permanent(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    perm = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        sign = (-1) ** (j % 2)
        perm += sign * A[0][j] * permanent(submatrix)
    return abs(perm)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    clauses = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i+1) for i in range(random.randint(1, n))]
        clauses.append(clause)

    M = [[0 for _ in range(n)] for _ in range(n)]
    for clause in clauses:
        sign = random.choice([-1, 1])
        for var in clause:
            if abs(var) <= n:
                M[abs(var)-1][clauses.index(clause)] += sign

    perm_matrix = [[0 for _ in range(n)] for _ in range(n)]
    det_matrix = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if (i+1) * (j+1) % 2 == 0:
                perm_matrix[i][j] = M[i][j]
            else:
                det_matrix[i][j] = -M[i][j]

    perm_val = permanent(perm_matrix)
    det_val = determinant(det_matrix)

    if perm_val == 0 or det_val == 0:
        return {
            "metric_name": "Multiplicity Gap",
            "metric_value": None,

```

```

        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Permanent or Determinant is zero"
    }

    perm_multiplicity = Fraction(perm_val, det_val)
    det_multiplicity = Fraction(det_val, det_val)

    return {
        "metric_name": "Multiplicity Gap",
        "metric_value": float(perm_multiplicity - det_multiplicity),
        "instances_tested": 1,
        "conjecture_holds": perm_multiplicity > det_multiplicity,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 30*3 + 1)) # Default to first 30 primes
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Multiplicity Gap does not hold\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with deterministic seed configuration

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	45163
2	preregistration	ollama_remote	qwen3:8b	0	0	24312
3	novelty	ollama_remote	qwen3:8b	0	0	20765
4	novelty	ollama_remote	qwen3:8b	0	0	15959
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16294
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15205
7	judge	ollama_remote	qwen3:8b	0	0	19709

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 157406 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/39e42593cacd.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/39e42593cacd.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/39e42593cacd.tar.gz (if generated)